

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Il laboratorio

Prima del laboratorio

Tre tracce, una a testa, sui progetti del corso

Nel Modulo 4 Claude Code lavora nel repository: legge, agisce, controlla. Il **laboratorio** gli dà un lavoro vero, con un piano, almeno un subagent e un controllo che può eseguire: 87 minuti in **due parti**, e questa è la prima, di 52

traccia	che cosa fa Claude Code	su quale progetto
a · M4-E02	**Una funzione nuova, con i test**: il filtro per apparato e per date della pagina Ticket; i test li scrive un subagent	il gestionale PHP
b · M4-E03	**Un modulo opaco: documentarlo e proporre il refactoring**, senza cambiarne una riga	il gestionale PHP
c · M4-E04	**Uno script operativo che gira davvero**: il controllo notturno, in PHP	il gestionale PHP
c · M4-E05	**Il rapporto quotidiano dei log**, l'altra variante: in Bash o in Python	la rete: una settimana di log

Tutte e tre sui progetti **sintetici** di CiaspoLink, l'operatore inventato del corso: ciò che Claude Code legge va ad **Anthropic**; se consentite l'uso dei dati per migliorare i modelli, può servire ad addestrarne di nuovi, e si conserva cinque anni invece di trenta giorni

A voce: quale traccia scegliete, e perché?

Le regole del laboratorio

Le stesse in ogni traccia, ciascuna con la frase di Anthropic che la regge

- **Il piano prima di scrivere codice**, in plan mode: «Plan mode tells Claude to research and propose changes without making them. … Except in interactive terminal sessions with bypass permissions available, edits stay blocked until you approve the plan.» Nella traccia b il piano è la proposta di refactoring, nella seconda parte
- **Almeno un subagent**, per una delle ragioni per cui ripaga: lavori indipendenti nello stesso momento e un revisore che non ha visto scrivere il codice (traccia a); un output lungo, come una ricerca ampia, tenuto fuori dalla conversazione principale (b); un revisore che legge e non modifica (c)
- **Un controllo che Claude può eseguire**: «If you give it something that comes back pass or fail, Claude can close that loop itself.» I test del gestionale, o lo script di controllo della traccia
- **Piccoli passi, un commit ciascuno**: «Do refactoring in small, testable increments»; e i team di Anthropic fanno commit di checkpoint regolari, per tornare indietro facilmente
- **Sonnet**, effort **medium**: «Sonnet handles most coding tasks well and costs less than Opus.» I subagent su **Haiku** dove il lavoro è semplice, e **riassunti brevi**: «Ask for short summaries back, so twenty results don’t fill your context anyway.»

L’impronta sul Pro

Quanto pesa una traccia, e che cosa lasciare alla seconda parte

traccia	turni, nella prova del corso	subagent
a · M4-E02	50	due: `autore-test` su Sonnet, una chiamata, e `revisore` su Haiku, due
b · M4-E03	48, di cui 21 per i test	due Explore, nella prima parte, sul modello della conversazione principale
c · M4-E04, M4-E05	77 e 63	uno: `revisore` su Haiku, sei chiamate nella prima e tre nella seconda

- Sessioni su **Sonnet**, effort **medium**. I test, i controlli e la verifica girano sul vostro computer, **senza chiamate** al modello; i subagent no: «What they cost you is mostly tokens». Il limite è lo stesso di claude.ai
- `/usage`: il costo della sessione e i limiti del piano; sul Pro divide l’utilizzo per skill, **subagent**, plugin e server MCP
- Prima di chiudere questa parte: un **commit** e `NOTA.md`, che cosa è fatto, che cosa manca, i comandi per riprendere. Una conversazione lunga, ripresa dopo un’interruzione lunga, ricostruisce la **cache**; sul Pro può comparire «Resume this conversation?»

Una traccia, un controllo, un subagent: il resto lo decidete voi.

Traccia a: una funzione nuova, con i test

M4-E02: la pagina Ticket del gestionale filtra anche per apparato e per date

- La pagina Ticket del gestionale filtra per stato, priorità, categoria e cliente; manca il filtro per **apparato** e per **intervallo di date**. `SPECIFICA.md` dice che cosa deve fare: sette requisiti, un esempio ciascuno, i casi di accettazione
- `autore-test` scrive i test dalla specifica, **in background**, mentre la sessione principale implementa; non legge i due file che intanto cambiano. «Running independent pieces of work at the same time»
- `revisore`, su Haiku e con soli Read, Grep e Glob, confronta il diff con la specifica prima del commit che chiude la funzione. «Getting a reviewer that hasn't seen how the code was written»
- Che cosa non danno: «You get parallelism or a fresh pair of eyes out of it, but not a smarter answer.» Tutti e due rispondono in un **formato fisso**
- Regole `deny` di `Edit` su `SPECIFICA.md`, su `.claude/agents/` e su `tools/php.sh`: si scrive il codice, non si cambiano la specifica, i subagent o lo strumento

Traccia a: una funzione nuova, con i test

M4-E02: la pagina Ticket del gestionale filtra anche per apparato e per date

```
# dalla cartella esercitazioni: il passo 1 del README di M4-E02
tools/prepara_esercizio.sh M4-E02
cd ~/corso-claude-code/M4-E02
tools/php.sh composer install -d progetti/gestionale-php
claude --permission-mode plan --model sonnet --effort medium
```

- Il **controllo** che Claude esegue dopo ogni passo, senza chiedervelo, per una regola `allow`: la suite PHPUnit, `tools/php.sh php progetti/gestionale-php/vendor/bin/phpunit -c progetti/gestionale-php`
- `tools/verifica.sh M4-E02`: la suite intera e i test nuovi **passano** sul vostro codice, e i test nuovi **falliscono** sul codice di partenza, dove la funzione manca. Il giudice va provato: «Run it against the original code to confirm it passes. Then run it against deliberately broken code to confirm it fails»
- Nella trascrizione del revisore, nessuno strumento che modifica: è una **garanzia** di Claude Code. I subagent li conta e li stampa, come **dato**: senza, «nessun subagent trovato», ma non vi boccia

Traccia b: un modulo opaco

M4-E03: documentarlo e proporre il refactoring, senza cambiarne una riga

- `legacy/chiusura_mese.inc.php` fa la chiusura di fine mese: una funzione sola, `chiusura_mese()`, di 278 righe, sei compiti in fila e, in testa, l'avvertenza «non spezzare in piu funzioni». Il modulo **non si modifica**: lo si mappa, lo si documenta, se ne fissa il comportamento, si propone come rifattorizzarlo
- Due subagent **Explore**, in parallelo nello stesso messaggio, con soli strumenti di lettura: chi chiama `chiusura_mese()`, e quali tabelle legge e scrive. Ciascuno torna con un **riassunto breve**, al massimo 12 righe. «Broad codebase searches go to the built-in Explore agent.» «Keeping long output, like a full test run or a wide search, out of your main conversation»
- La regola deny di `Edit(/progetti/gestionale-php/legacy/**)`, ancorata al progetto: «Deny rules block in every mode, including bypassPermissions.» Copre gli strumenti di Claude Code, non ogni processo: per questo la verifica controlla anche, con **git**, che `legacy/` sia rimasto com'era

Traccia b: un modulo opaco

M4-E03: documentarlo e proporre il refactoring, senza cambiarne una riga

```
# dalla cartella esercitazioni: il punto di partenza e il passo 1
tools/prepara_esercizio.sh M4-E03
cd ~/corso-claude-code/M4-E03
tools/php.sh composer install -d progetti/gestionale-php
claude --model sonnet --effort medium
```

- La **documentazione**, `docs/chiusura_mese.md`, con sei sezioni fisse, da «Che cosa fa» a «Rischi»: lo standard contro cui si controlla, «check if the documentation follows our project standards». I **test di caratterizzazione**, `tests/Integrazione/ChiusuraMeseTest.php`: fissano ciò che la chiusura fa adesso, anche dove sembra strano
- Nella seconda parte, in **plan mode**, la proposta: `PROPOSTA.md`, un titolo «`## Passo N`» per passo, ciascuno con la sua riga «`Controllo:`». «Do refactoring in small, testable increments»
- `tools/verifica.sh M4-E03`: le sei sezioni; i passi, ognuno con il suo controllo; i test **verdi** sul modulo com'era e **rossi** su una copia che la verifica rompe apposta, il giudice provato; `legacy/` com'era nel primo commit; i file che i passi chiedono, nei **commit**

Traccia c: uno script che gira davvero

Due varianti, una a scelta: M4-E04 nel gestionale, M4-E05 sulla rete

- **M4-E04, il gestionale**: `bin/controllo-notturno.php`, in PHP; ogni notte, **quattro controlli** e due conteggi
- **M4-E05, la rete**: `rapporto_giornaliero.py`, o `.sh` in Bash; ogni mattina, i **disservizi** del giorno prima, cinque tipi
- Prima di lasciarlo lavorare da solo, le **cinque domande** di Anthropic, e le risposte nello spazio di lavoro:

1 Sa com'è il lavoro **finito**?	`SPECIFICA.md`: le opzioni, le uscite 0, 1 e 2, stdout e stderr
2 Può **controllare** il proprio lavoro?	`python3 controlla_notturno.py`, o `python3 controlla_rapporto.py`: risponde «passa» o «non passa»
3 Che cosa può **toccare**?	lo script: `deny` di `Edit` sul controllo, su `SPECIFICA.md`, su `.claude/agents/` e su `src/`, `legacy/` e `database/` del gestionale, o su tutto `progetti/rete/`
4 Che cosa lo **ferma**?	qui voi, in sessione: Esc lo ferma e tiene il lavoro fatto; senza nessuno davanti, un limite: `--max-turns`, `--max-budget-usd`, un limite di spesa
5 Dove va il **risultato**, e chi lo **approva**?	un commit per passo e `NOTA.md`: li approvate voi

- Il subagent che ripaga: `revisore`, su Haiku e con soli Read, Grep e Glob, controlla il contratto dello script prima del commit che lo chiude: «a reviewer that can read but not edit». «Now it's a reviewer that can't edit anything», «enforced by Claude Code»

Traccia c: uno script che gira davvero

Due varianti, una a scelta: M4-E04 nel gestionale, M4-E05 sulla rete

```
# dalla cartella esercitazioni, M4-E04: il passo 1 del README
tools/prepara_esercizio.sh M4-E04
cd ~/corso-claude-code/M4-E04
tools/php.sh composer install -d progetti/gestionale-php
tools/php.sh php progetti/gestionale-php/bin/inizializza-db.php
claude --permission-mode plan --model sonnet --effort medium
# oppure M4-E05: il passo 1 del suo README
tools/prepara_esercizio.sh M4-E05
cd ~/corso-claude-code/M4-E05
claude --permission-mode plan --model sonnet --effort medium
```

- ⚖ `tools/verifica.sh M4-E04`: lo script sui dati di esempio, con `--giorno=2026-09-13`, esce con **1** e trova le incoerenze inserite apposta, né una di meno né una di più; sugli stessi dati senza incoerenze esce con **0**
- ⚖ `tools/verifica.sh M4-E05`: martedì 8 settembre esce con **1**, e fra le sue righe c'è il disservizio che la verifica conosce, campo per campo; in un giorno tranquillo esce con **0**, senza nessuna riga
- 🔒 In tutte e due, nella trascrizione del revisore nessuno strumento che modifica: è una **garanzia** di Claude Code

Al lavoro

La prima parte del laboratorio, con il README della vostra traccia

- I README, nel repository delle esercitazioni: `m4_claude_code_agenti/s3_agente_nel_repository/m4_e2_laboratorio/`, poi `M4-E02_funzione_con_test/` · `M4-E03_modulo_opaco/` · `M4-E04_script_operativo/` · `M4-E05_rapporto_log/`. Lì i passi e i **prompt**, da incollare uno alla volta
- La prima volta che aprite `claude` nello spazio di lavoro, Claude Code vi chiede se vi **fidate della cartella**: sì. Le regole `allow`, come quella del controllo, valgono **solo dopo**; le `deny`, subito

I traguardi, non i minuti

1 il piano approvato	2 il primo controllo verde	3 il riassunto del subagent letto	4 un commit per passo
leggetelo, poi «Yes, manually approve edits»: le modifiche, una per una. Nella traccia b, il piano è della seconda parte	PHPUnit dice OK; lo script di controllo, nell'ultima riga, «uscita: 0»	nel suo formato fisso; nella traccia b, i riassunti dei due Explore	`git add`, poi `git commit -m`, un comando per volta: il messaggio lo leggete voi

- Una riga che unisce più comandi: **No**, un comando per volta. Se il primo `git commit` si ferma con «Please tell me who you are», git non sa chi firma i commit: il rimedio è nel README, in «Se qualcosa va storto»

Prima di chiudere questa parte

Un commit e una nota: la seconda parte del laboratorio riparte da lì

```
# nella sessione: il prompt 4 del README, NOTA.md e il suo commit
/exit                    # la prima parte finisce qui
git log --oneline        # l'ultimo commit: quello di NOTA.md
```

- `NOTA.md`, nella cartella dello spazio di lavoro: che cosa è **fatto**, che cosa **manca**, i comandi per **riprendere**. Il commit lo approvate voi
- La seconda parte del laboratorio riparte da **questa nota**, in una sessione nuova di Claude Code: «Each new session starts with a fresh context window, without the conversation history from previous sessions». La nota è piccola; la conversazione di questa parte, no
- Riprendere una conversazione lunga dopo un'interruzione lunga: la richiesta successiva ricalcola tutto l'input e ricostruisce la cache, e sul Pro può comparire «Resume this conversation?»
- Non avete finito? Va bene: la nota dice che cosa manca, e i commit tengono ciò che passa il controllo